# ECEN/CSEN 123 Lab 7 Report

Tuesday 2:15 - 5:00

Austin Petersen, Dylan Thornburg, Madeline Follosco

## I. Objectives

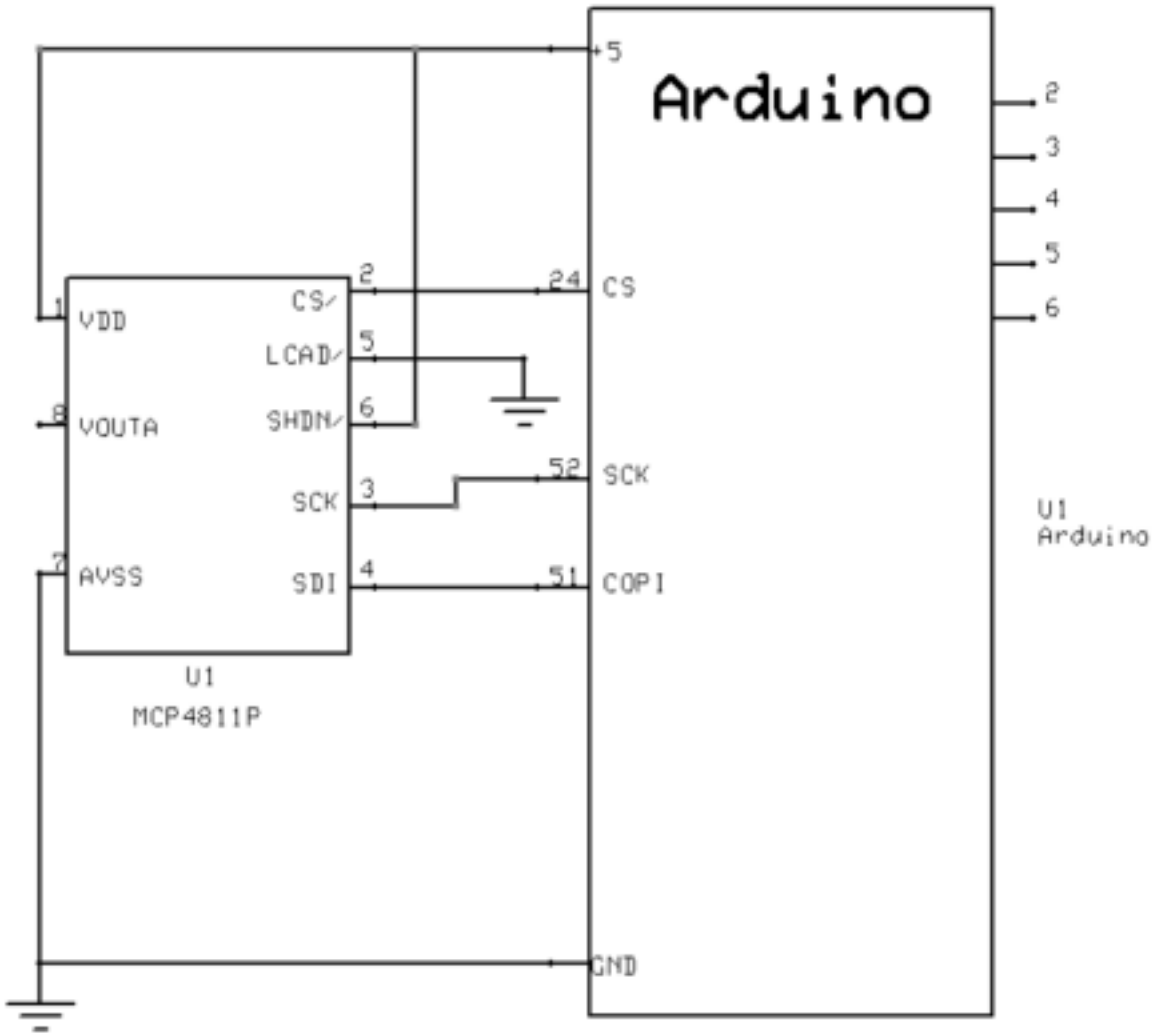
- Learn how to program an SPI device and use the SPI library

• Learn to generate signals with a DAC
• Learn to use an Audio Amplifier

## III. Lab Procedure 1

### *Experiment 1:*

Connect the MCP4811 D/A converter to your Arduino as shown below.




Connect a voltmeter between MCP4811 pin 8 and ground.

Copy the sketch SPI_DAC_BARE from the lab handouts directory. It should show you how to initialize the SPI interface pins and includes a function to send a DAC value to the chip.

Call the function with an appropriate value to set the DAC output voltage to 2.0 volts. (Demo#1)

| **CODE** |
|---|

```
#include <SPI.h>

#define COPI 51
#define SCK 52
#define CS 24


/* dacval must be between 0 and 1023 or there will be an error */
/* Vref is set at 4.096V */

void DACout(unsigned int dacval) {
  dacval = dacval*4 + 0x1000;
  digitalWrite(CS, LOW);
  SPI.transfer16(dacval);
  digitalWrite(CS, HIGH);              //
}



void setup() {
  SPI.begin();   // initiate SPI as controller
  SPI.beginTransaction(SPISettings(4000000, MSBFIRST, SPI_MODE0)) ;
  pinMode(CS, OUTPUT);
  digitalWrite(CS, HIGH);

  DACout(500);
}

void loop() {
}
```

### *Experiment 2:*

Next, copy the sketch Sine_Example_table8 from the lab handouts folder. Modify it so that it does not send data out to the serial port but instead sends data to the DAC.
Adjust the program parameters to generate a 40Hz sine wave with 60 steps per cycle. Display this sine wave on the scope. (Demo #2)

**CODE**

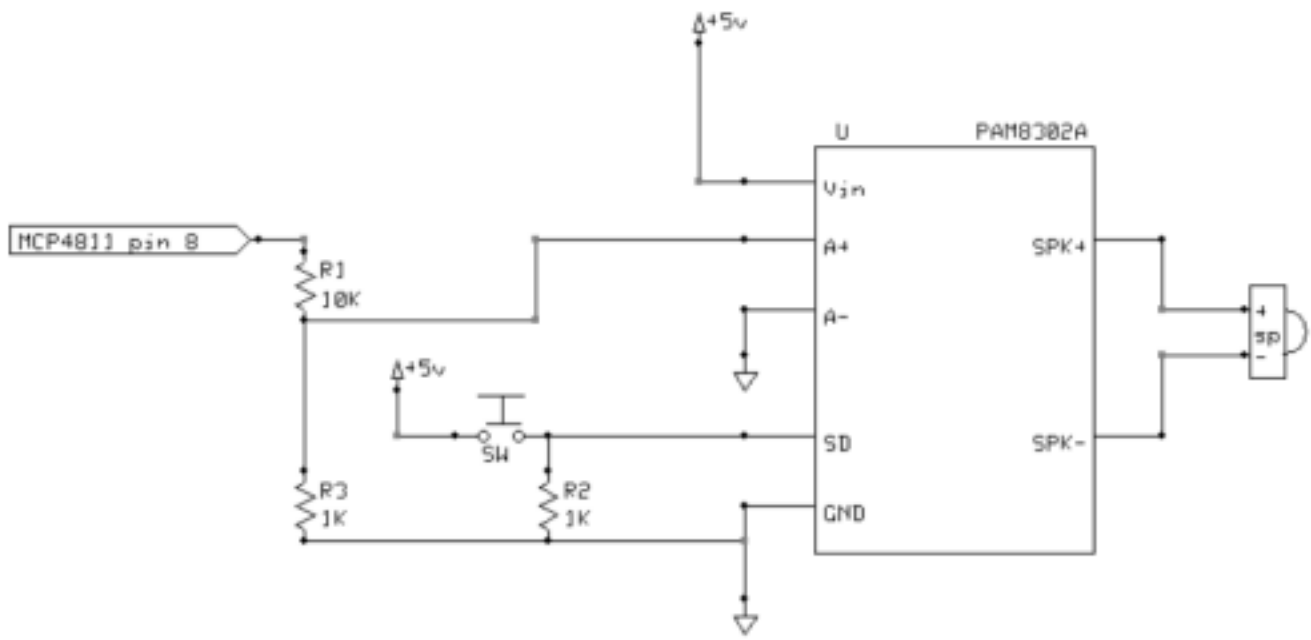
```
#include <SPI.h>
#define PRINTRATE 416 // (1 / (60 samples per cycle * 40hz)) in us
#define PHASEINC ((64 * 256)/60)  /* 60 Samples per full cycle */

int inval;
int phase;
long lastDACTime;
int waveTable[256];

#define CS 24

/* dacval must be between 0 and 1023 or there will be an error */
/* Vref is set at 4.096V */

void DACout(unsigned int dacval) {
  dacval = dacval*4 + 0x1000;
  digitalWrite(CS, LOW);
  SPI.transfer16(dacval);
  digitalWrite(CS, HIGH);
}


int wave(int phase) {
    if ((phase < 0) || (phase >= (256 * 64))) {
     return 0;
   }
   return waveTable[phase/64];
}


void setup() {
  phase = 0;
  lastDACTime = 0;
  init_waveTable();
  SPI.begin();   // initiate SPI as controller
```

```
  SPI.beginTransaction(SPISettings(4000000, MSBFIRST, SPI_MODE0)) ;
  pinMode(CS, OUTPUT);
  digitalWrite(CS, HIGH);
}

void loop() {
  if ((micros()-lastDACTime) > PRINTRATE) {
    DACout(wave(phase));
    lastDACTime = lastDACTime + PRINTRATE;
    phase = phase + PHASEINC;
    if (phase >= (256 * 64)) {
      phase = phase - (256*64);
    }
  }
}
```

### *Experiment 3:*

1) Copy your program and modify the copy so that it outputs a sine wave at 300Hz with 20 steps/cycle. Also, simultaneously output a square wave on Arduino pin 2 at the same frequency. View both on the scope.
2) Connect the audio amplifier module as shown below. The module can fit into your protoboard. Connect the speaker as shown[1]. (Note – you cannot see the sine waves on the amplifier output)
3) When you push the button, you should hear the 300Hz tone.
4) Switch the amplifier input (into R1) from the MCP4811 to pin 2 of the Arduino. You should hear a similar tone. Describe both tones and any differences between them in your report.

5) Borrow a large white speaker from me. Repeat steps 3 and 4. Document any differences in what you hear. (Demo #3 – 300Hz tone on the large speaker)
6) See if you can increase the steps per cycle and still hit 300Hz. See if the tone improves.




[1]If you have 2 speakers, just connect one.

**CODE**

```
#include <SPI.h>
#define PRINTRATE 167 // (1 / (20 samples per cycle * 300hz)) in us
#define PHASEINC ((64 * 256)/20)  /* 20 Samples per full cycle */

int inval;
int phase;
long lastDACTime;
int waveTable[256];

#define CS 24
#define SQ_WAVE_PIN 2

/* dacval must be between 0 and 1023 or there will be an error */
/* Vref is set at 4.096V */

void DACout(unsigned int dacval) {
```

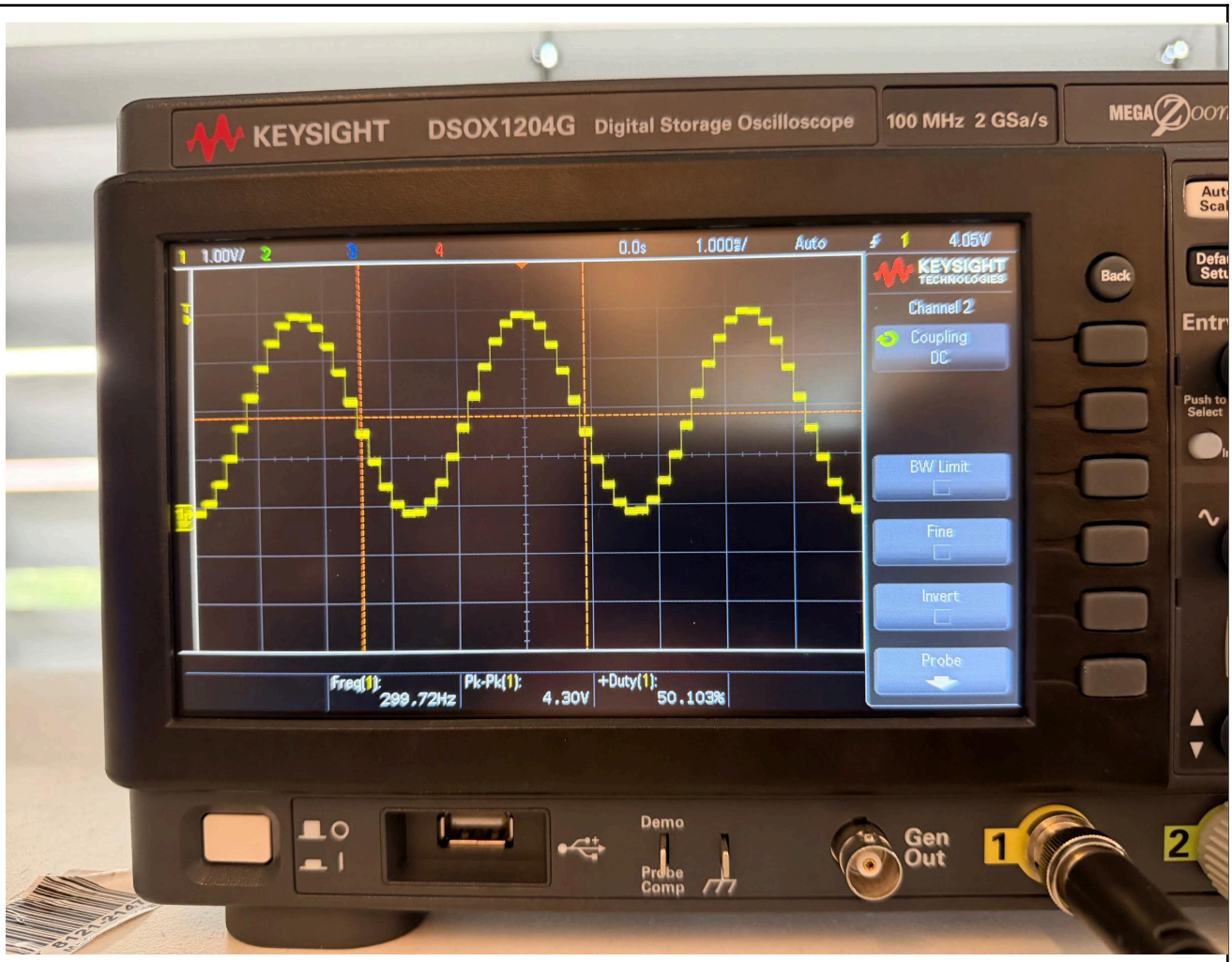
```
  dacval = dacval*4 + 0x1000;
  digitalWrite(CS, LOW);
  SPI.transfer16(dacval);
  digitalWrite(CS, HIGH);
}


int wave(int phase) {
    if ((phase < 0) || (phase >= (256 * 64))) {
     return 0;
   }
   return waveTable[phase/64];
}


void setup() {
  phase = 0;
  lastDACTime = 0;
  init_waveTable();
  SPI.begin();   // initiate SPI as controller
  SPI.beginTransaction(SPISettings(4000000, MSBFIRST, SPI_MODE0)) ;
  pinMode(CS, OUTPUT);
  digitalWrite(CS, HIGH);

  pinMode(SQ_WAVE_PIN, OUTPUT);
  digitalWrite(SQ_WAVE_PIN, LOW);
}

void loop() {
  if ((micros()-lastDACTime) > PRINTRATE) {
    DACout(wave(phase));

    digitalWrite(SQ_WAVE_PIN, (phase < ((256 * 64) / 2)) ? HIGH : LOW);
//first half cycle high, second low

    lastDACTime = lastDACTime + PRINTRATE;
    phase = phase + PHASEINC;
    if (phase >= (256 * 64)) {
```

```
      phase = phase - (256*64);
    }
  }
}
```

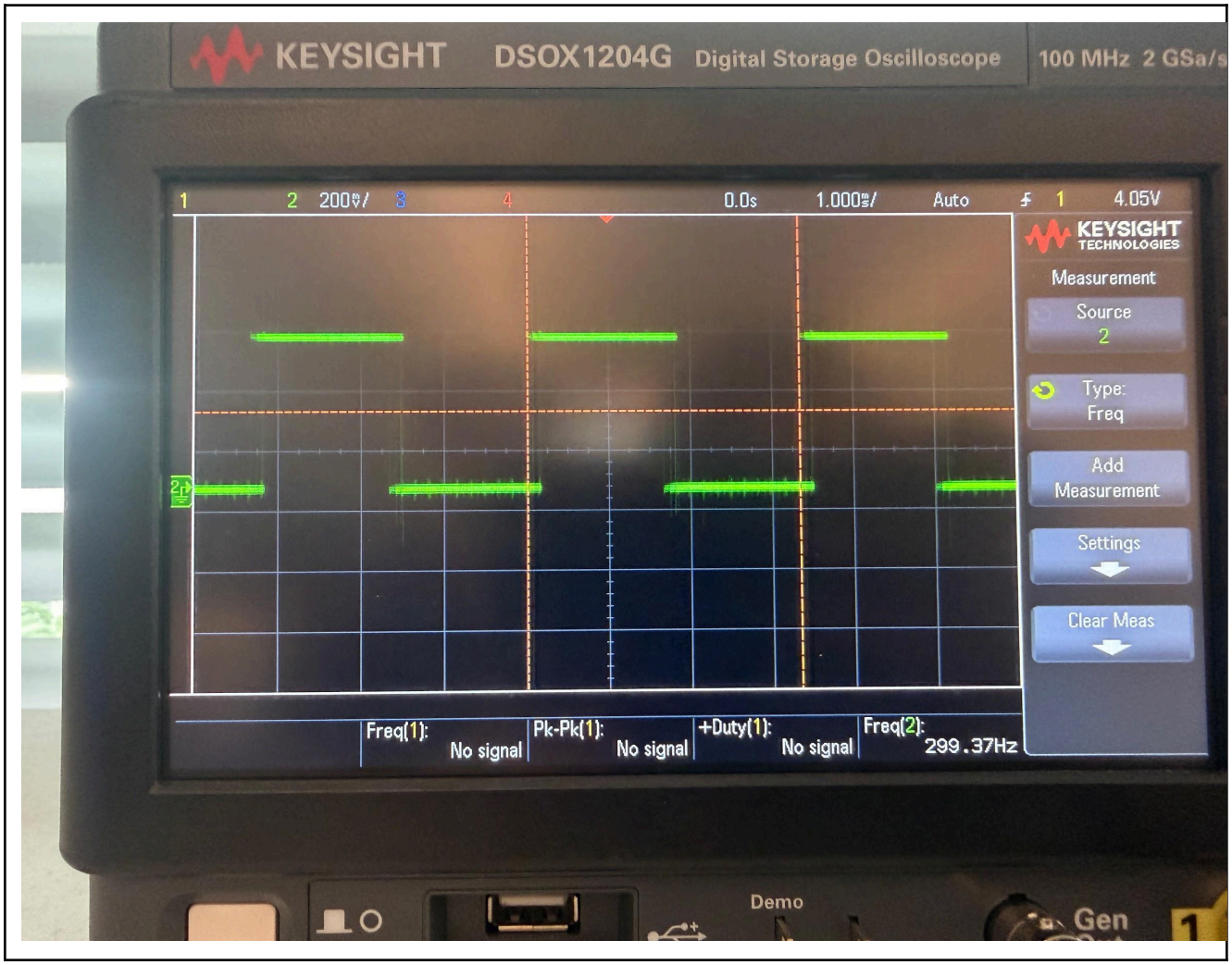
Observations:

| | Step 3<br>amplifier input (into R1) from MCP4811 | Step 4 (output a square wave)<br>amplifier input (into R1) from pin 2 of the Arduino |
|---|---|---|
| Original Speaker | Smoother, duller sound | Higher pitched, scratchy |
| White Speaker | Loud lower pitch, smoother | Loud high pitched and scratchy sound |

| |
|---|
| Sin Wave - 300 Hz |




Square Wave - 300 Hz




***Experiment 4:***

1) Copy your program and modify the copy so that it includes a function:

void mytone(int freq, long tonelength) // frequency in Hz, length in µs

This function will play a note at the frequency provided for the time specified.

The following frequencies approximately correspond to the following notes:

| | | | |
|---|---|---|---|
| A | 55 | 110 | 220 |
| B | 62 | 123 | 247 |
| C | 65 | 130 | 262 |
| D | 73 | 147 | 294 |

| | | | |
|---|---|---|---|
| E | 82 | 165 | 330 |
| F | 87 | 175 | 349 |
| G | 98 | 196 | 392 |

Assume a quarter note is 400,000 µs.

Test your routine by having it play some notes.

Then modify your program to play Mary had a little lamb over and over. (at least the first two stanzas)
http://www.enchantedlearning.com/music/sheetmusic/maryhadalittlelamb.shtml

If you can't read the music – find someone who can.

It may sound better if there is a slight pause between notes.

Demo #4

**CODE**

```
#include <SPI.h>
#define PRINTRATE 167 // (1 / (60 samples per cycle * 40hz)) in us
#define PHASEINC ((64 * 256)/20)  /* 20 Samples per full cycle */
#define QUARTER_NOTE 400000L
#define CS 24
#define SQ_WAVE_PIN 2

#define A1 55
#define A2 110
#define A3 220
#define A4 440
#define B1 62
#define B2 123
#define B3 247
#define B4 494
#define C1 65
#define C2 130
#define C3 262
#define C4 524
```

```
#define D1 73
#define D2 147
#define D3 294
#define D4 588
#define E1 82
#define E2 165
#define E3 330
#define E4 660
#define F1 87
#define F2 175
#define F3 349
#define F4 698
#define G1 98
#define G2 196
#define G3 392
#define G4 784


int inval;
int phase;
long lastDACTime;
int waveTable[256];



/* dacval must be between 0 and 1023 or there will be an error */
/* Vref is set at 4.096V */

void DACout(unsigned int dacval) {
  dacval = dacval*4 + 0x1000;
  digitalWrite(CS, LOW);
  SPI.transfer16(dacval);
  digitalWrite(CS, HIGH);
}


int wave(int phase) {
    if ((phase < 0) || (phase >= (256 * 64))) {
```

```
      return 0;
    }
    return waveTable[phase/64];
}

void mytone (int freq, long tonelength){ // all in us
  if (freq <=0 || tonelength <=0) return;
  unsigned long halfPeriod = 1000000UL / (2UL*(unsigned long)freq);
  unsigned long start = micros();
  while ((long)(micros() - start) < tonelength) {
    digitalWrite(SQ_WAVE_PIN, HIGH);
    delayMicroseconds(halfPeriod);
    digitalWrite(SQ_WAVE_PIN, LOW);
    delayMicroseconds(halfPeriod);
  }
}

//if pausing is needed
void pause(long pause){
  digitalWrite(SQ_WAVE_PIN, LOW);
  delayMicroseconds(pause);
}

void setup() {
  phase = 0;
  lastDACTime = 0;
  init_waveTable();
  SPI.begin();   // initiate SPI as controller
  SPI.beginTransaction(SPISettings(4000000, MSBFIRST, SPI_MODE0)) ;
  pinMode(CS, OUTPUT);
  digitalWrite(CS, HIGH);

  pinMode(SQ_WAVE_PIN, OUTPUT);
  digitalWrite(SQ_WAVE_PIN, LOW);
}

void loop() {
  /*if ((micros()-lastDACTime) > PRINTRATE) {
```

```
    DACout(wave(phase));

    digitalWrite(SQ_WAVE_PIN, (phase < ((256 * 64) / 2)) ? HIGH : LOW);
//first half cycle high, second low

    lastDACTime = lastDACTime + PRINTRATE;
    phase = phase + PHASEINC;
    if (phase >= (256 * 64)) {
      phase = phase - (256*64);
    }
  } */
  mary();
  delay(3000);
}

void mary(){
  mytone(B2, QUARTER_NOTE);
  pause(20000);
  mytone(A2, QUARTER_NOTE);
  pause(20000);
  mytone(G1, QUARTER_NOTE);
  pause(20000);
  mytone(A2, QUARTER_NOTE);
  pause(20000);

  mytone(B2, QUARTER_NOTE);
  pause(20000);
  mytone(B2, QUARTER_NOTE);
  pause(20000);
  mytone(B2, 2 * QUARTER_NOTE);
  pause(40000);

  mytone(A2, QUARTER_NOTE);
  pause(20000);
  mytone(A2, QUARTER_NOTE);
  pause(20000);
  mytone(A2, 2 * QUARTER_NOTE);
  pause(40000);
```

```
  mytone(B2, QUARTER_NOTE);
  pause(20000);
  mytone(D2, QUARTER_NOTE);
  pause(20000);
  mytone(D2, 2 * QUARTER_NOTE);
  pause(40000);

  mytone(B2, QUARTER_NOTE);
  pause(20000);
  mytone(A2, QUARTER_NOTE);
  pause(20000);
  mytone(G1, QUARTER_NOTE);
  pause(20000);
  mytone(A2, QUARTER_NOTE);
  pause(20000);

  mytone(B2, QUARTER_NOTE);
  pause(20000);
  mytone(B2, QUARTER_NOTE);
  pause(20000);
  mytone(B2, QUARTER_NOTE);
  pause(20000);
  mytone(B2, QUARTER_NOTE);
  pause(20000);

  mytone(A2, QUARTER_NOTE);
  pause(20000);
  mytone(A2, QUARTER_NOTE);
  pause(20000);
  mytone(B2, QUARTER_NOTE);
  pause(20000);
  mytone(A2, QUARTER_NOTE);
  pause(20000);

  mytone(G1, 4 * QUARTER_NOTE);
  pause(80000);
}
```